



**Документация на курсова работа по
Програмиране за мобилни
устройства**

Александър Евгениев Ганчев

ФКСТ, КСИ, група 39а

№ 121223237



Съдържание

1. Увод.....	2
2. Анализ на съществуващи разработки.....	3
3. Проектиране на системата.....	5
4. Софтуерна реализация.....	10
5. Потребителско ръководство и тестване.....	13
6. Приложение.....	15
7. Заключение.....	18
8. Литература.....	19



1. Увод

В ежедневието магазини, пекарни и заведения често оставят годни за консумация храни, които не успяват да се продадат. В същото време доста хора търсят по-достъпни начини за покупка на храна. Именно това се опитва да реши приложението.

То представлява мобилно iOS приложение, разработено със SwiftUI, което свързва два типа потребители - магазини и клиенти. Магазините създават обяви за налична храна с намалена цена и задават времеви интервал за получаване. Клиентите разглеждат предложенията в списък или на карта, филтрират резултатите, резервират пакет, а след това получават код за потвърждение при вземане. За реализацията на удостоверяването и съхранението на данни са използвани Firebase Authentication и Cloud Firestore [2], [3].

Функционалностите, които приложението има, могат да бъдат обобщени така: 1) реализиране на регистрация, вход и верификация по имейл 2) разделяне на потребителите на роли 3) управление на жизнения цикъл на една обява 4) създаване и промяна на резервации 5) филтриране и сортиране на предложения 6) представяне на статуса на поръчките и въвеждане на код за получаване. От гледна точка на курсовата работа проектът има за цел не само да демонстрира работещ софтуерен проект, но и да покаже правилен подход към архитектура, моделиране на данни, потребителски интерфейс и тестване [9].



2. Анализ на съществуващи разработки

Преди създаването на приложението, разгледах няколко различни приложения, които имат същата цел. Сред най-разпознаваемите платформи в тази област са Too Good To Go, Olio и Karma. Въпреки че целите им са сходни, техните модели на работа имат различен фокус, което е полезно при създаването на приложението [6], [7], [8].

Too Good To Go е ориентирано основно към резервиране и закупуване на пакети с излишна храна от магазини и заведения, като потребителят избира предложение, запазва го и го получава в зададен интервал [6]. Olio комбинира елементи на споделяне и размяна на безплатна храна и други продукти [7]. Karma също е насочено към продажбата на непродадена храна с намаление, но има и по-широк набор от функции за управление на продажбите и наличностите от страна на бизнеса [8].

За нуждите на курсовата работа не е търсено пълно копиране на която и да е съществуваща система. Вместо това са използвани основни функции: наличие на времеви интервал за получаване, локация на предложенията върху карта, ясно разделение на ролите и минимален брой действия за изпълнение на една резервация. На тази база е разработено опростено, но реалистично приложение.

2.1 Сравнителен преглед

Система	Основен модел	Карта/локация	Резервация	Потвърждение
Too Good To Go	Пакети с излишна храна от търговски обекти	Да	Да	В рамките на приложението
Olio	Споделяне на храна и предмети наблизо	Да	Да	Организация на вземането между потребители
Karma	Продажба на излишна храна с намаление	Да	Да	Код за получаване
FoodSaver	Оферти от магазин към клиент без плащане в приложението	Да	Да	Код за получаване при магазина



2.2 Изводи от анализа

От анализа на съществуващите разработки следват няколко важни извода. Първо, наличието на две ясно разграничени роли е задължително, защото поведението на магазин и клиент е различно. Второ, визуализацията на предложенията трябва да бъде едновременно под формата на списък и карта, за по-лесно намиране на близки оферти.



3. Проектиране на системата

Използваната архитектура е MVVM, защото има разделяне между визуализацията, състоянието на интерфейса и бизнес логиката. Така се улеснява работата с динамично обновяване на екрана при промяна на данните [1], [9].

3.1 Функционални изисквания

1) Системата трябва да позволява регистрация, вход и изход на потребители чрез имейл и парола.

2) След регистрация потребителят трябва да избере роля - клиент или магазин.

3) Магазинът трябва да може да създава, редактира, активира, деактивира и изтрива свои обяви.

4) Клиентът трябва да може да търси, филтрира и сортира активни предложения.

5) Клиентът трябва да може да резервира предложение, ако то е активно, неизтекло и има количество.

6) Клиентът трябва да може да откаже активна резервация.

7) Магазинът трябва да може да потвърди получаване чрез код за вземане.

8) Системата трябва да поддържа разделяне на поръчките по статус - резервирани, изпълнени и отказани.

3.2 Нефункционални изисквания

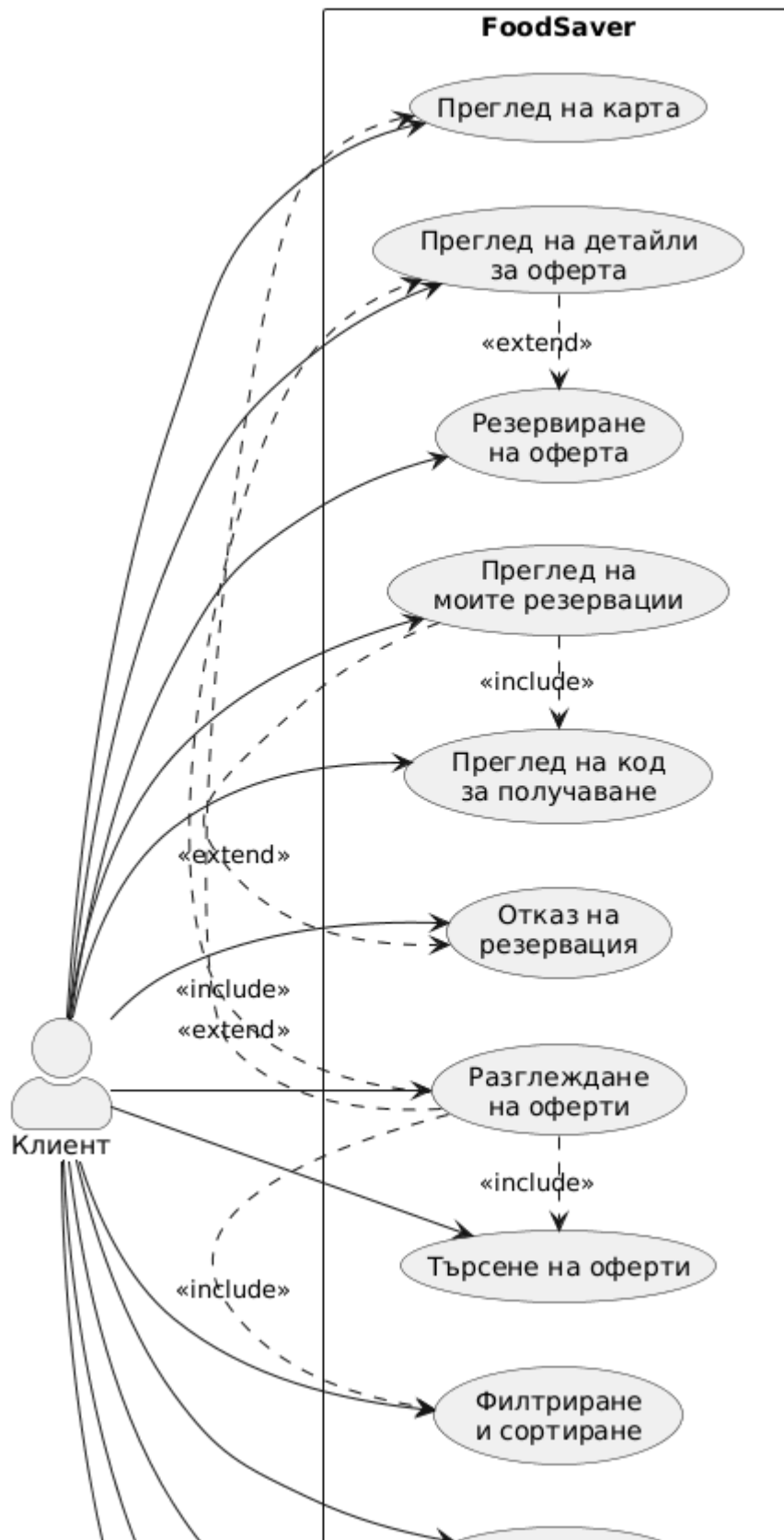
1) Интерфейсът трябва да бъде изцяло на български език.

2) Системата трябва да работи коректно при невалидни входни данни и да показва разбираеми съобщения за грешка.

3) Операциите за резервация и отказ трябва да се изпълняват като едно цяло, за да се избегне несъответствие между поръчки и наличности.

4) Достъпът до данните в Cloud Firestore трябва да бъде защитен и определян според ролята на потребителя [4].

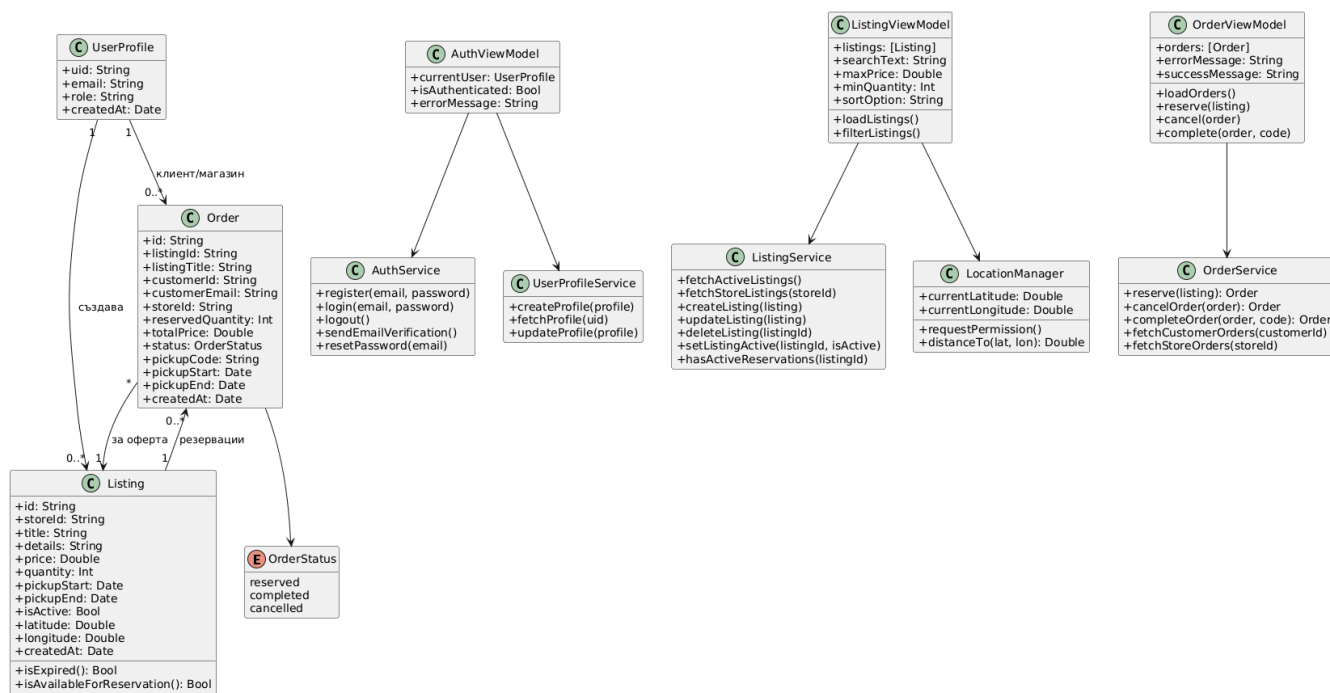
3.3 Use case диаграма





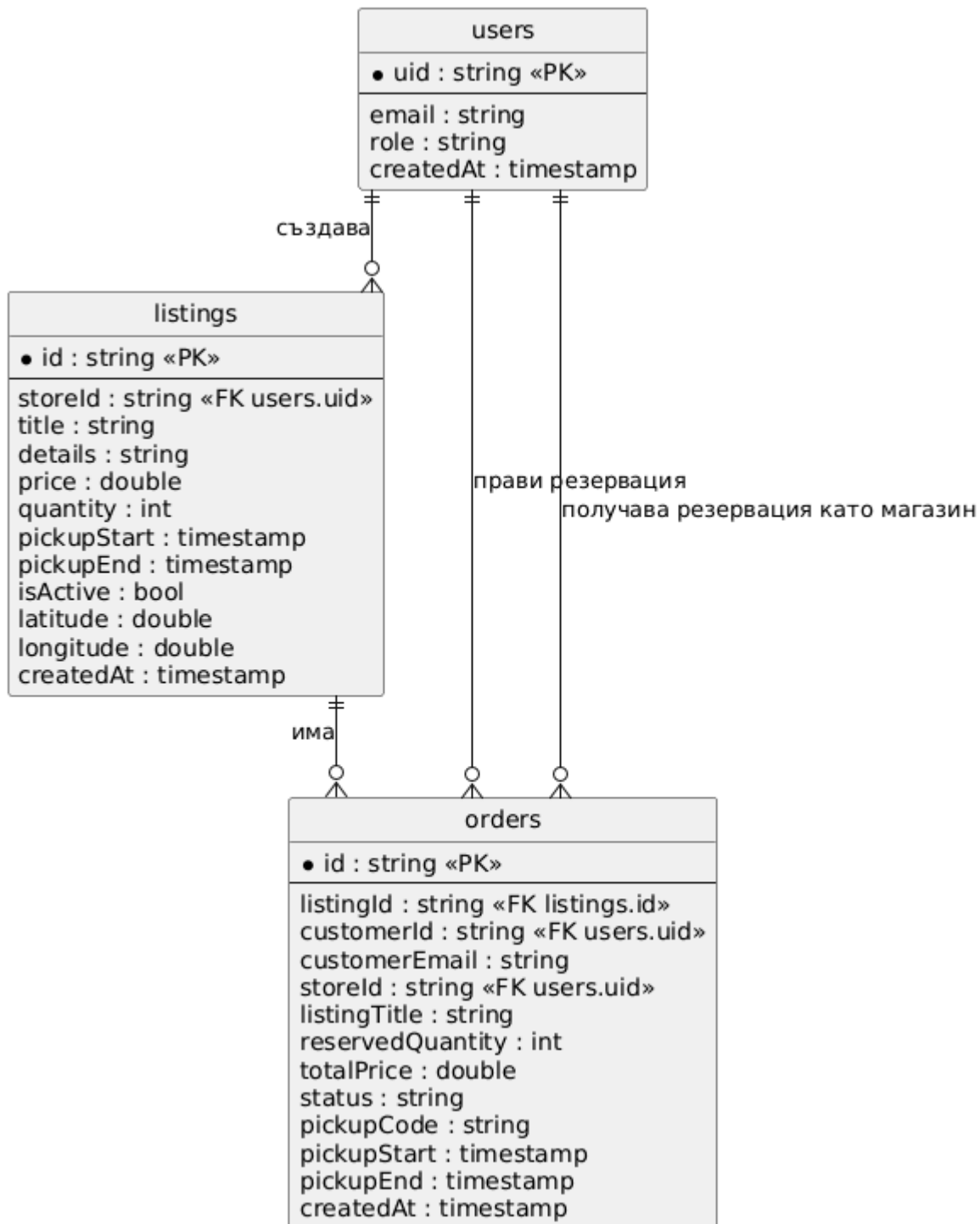


3.4 Клас диаграма





3.5 ER диаграма



3.6 Архитектура и основни компоненти

В архитектурно отношение системата е разделена на четири основни слоя. Първият слой е потребителският интерфейс (Views), реализиран чрез SwiftUI. Вторият слой са ViewModel класовете, които поддържат динамичното състояние на view-тата и осъществяват връзката между



интерфейса и бизнес логиката. Третият слой са Service класовете, които реализират достъп до Firebase Authentication и Cloud Firestore. Четвъртият слой е моделът на данните - структури за потребителски профил, обява и резервация.

Така кодът се чете по-лесно, промени в бизнес логиката не налагат директни промени във всеки интерфейс, а тестването на основните функции се извършва на ниво Service и ViewModel класове. В RootView се реализира маршрутизацията според състоянието на удостоверяване, потвърждения имейл и избраната роля, което опростява логиката на приложението.

3.7 Модели на данните

Колекция	Основни полета	Предназначение	Достъп
users	uid, email, role, createdAt	Профил и тип на потребителя	Собственик
listings	id, storeId, title, details, price, quantity, pickupStart, pickupEnd, isActive, latitude, longitude	Обяви за налична храна	Четене от всеки потребител промяна от магазин (собственик)
orders	id, listingId, customerId, storeId, totalPrice, status, pickupCode, pickupStart, pickupEnd	Резервации и проследяване на статуса	Клиент/магазин според участието в поръчката

3.8 Основни потребителски сценарии

Клиент: регистрация -> потвърждение на имейл -> избор на роля -> разглеждане на обяви -> филтриране -> резервация -> получаване на код -> отказ или успешно получаване

Магазин: регистрация -> потвърждение на имейл -> избор на роля -> създаване на обява -> редакция/деактивация -> преглед на резервации -> предаване на поръчки



4. Софтуерна реализация

Входната точка е FoodSaverApp, където се конфигурира Firebase и се зарежда RootView. Интерфейсът използва настройка bg_BG, което позволява визуализиране на текстове на български език. За разработката са използвани библиотеките FirebaseCore, FirebaseAuth и FirebaseFirestore, добавени чрез Swift Package Manager [2], [3].

4.1 Организация на проекта

Папка	Съдържание	Роля в приложението
Models	Listing, Order, UserProfile	Описват структурата на данните и някои изчислими свойства
Services	AuthService, UserProfileService, ListingService, OrderService	Изолират достъпа до Firebase и бизнес логиката
ViewModels	AuthViewModel, ListingViewModel, OrderViewModel	Управляват състоянието на екраните и обработват събития
Views	Auth, Customer, Store, Orders, Root, Shared	Представят визуалните екрани и взаимодействието с потребителя
Managers	LocationManager	Осигурява достъп до местоположението и комуникация с Core Location

4.2 Удостоверяване и профили

AuthService реализира операциите по регистрация, вход, изход, изпращане на имейл за потвърждение и възстановяване на парола. След успешен вход RootView проверява дали имейлът е потвърден и дали потребителят вече има профил в базата данни users. Ако профил липсва (регистрация), се показва екран за избор на роля [2].

4.3 Управление на обяви

ListingService реализира създаване, редактиране, изтриване и промяна на активността на обявите. При запис се валидират цена, количество, координати и времеви прозорец за получаване. Активните обяви за клиента се извличат чрез заявка по полето isActive и се филтрират чрез функцията isAvailableForReservation. За магазина е реализирано разделяне на активни, неактивни/изчерпани и изтекли обяви. Важно е ограничението при редактиране, ако за дадена обява вече има активни резервации, могат да се променят само полета като описанието, за да не се наруши вече поет ангажимент към клиента.



4.4 Резервации и код за получаване

OrderService съдържа най-съществената бизнес логика. При резервиране се използва Firestore транзакция, която прочита текущото състояние на обявата, проверява наличността, намалява количеството и създава поръчка със статус reserved. В прозореца на поръчката се записва и кратък код за получаване. По този начин магазинът може да потвърди, че клиентът действително е взел пакета, без да е необходима платежна система или QR код. При отказ на резервация друга транзакция връща количеството към съответната обява и променя статуса на поръчката на cancelled.

Този транзакционен подход е използван, защото предотвратява ситуации, при които две действия почти едновременно водят до неконсистентни данни. Например, ако две устройства се опитат да резервират последната налична бройка, транзакцията гарантира, че ще успее само операцията, при която данните все още са актуални към момента на записване [3], [4].

4.5 Търсене, филтриране и карта

ListingViewModel реализира търсене по заглавие и описание, филтриране по максимална цена и минимално количество, както и сортиране по три критерия - най-скоро взимане, най-ниска цена и най-близо. За сортиране по близост се използва LocationManager, който извлича текущото местоположение на устройството чрез Core Location [5]. MapKit се използва за визуализация на предложенията върху карта, като за всяка обява с валидни координати се поставя маркер.

4.6 Защита на данните и правила за достъп

Cloud Firestore Security Rules ограничават достъпа до документите според идентичността и ролята на потребителя. Само магазин може да създава и редактира собствените си обяви, само клиентът може да създава своя резервация, а поръчките се четат само от участващия клиент и съответния магазин [4].



5. Потребителско ръководство и тестване

5.1 Работа с приложението от клиент

1) Потребителят създава акаунт чрез имейл и парола и потвърждава имейла си.

2) При първо влизане избира роля „Клиент“.

3) Отваря секцията „Разгледай оферти“, където вижда активните предложения.

4) По желание използва търсене, филтри и сортиране или преглежда предложенията на карта.

5) Отваря конкретна обява, проверява цена, интервал за получаване и количество и резервира пакет.

6) Създадената резервация се вижда в „Моите резервации“, където е достъпен и кодът за получаване.

7) При необходимост клиентът може да откаже активна резервация преди получаване.

5.2 Работа с приложението от магазин

1) Потребителят се регистрира, потвърждава имейла си и избира роля „Магазин“.

2) От таблото за управление създава нова обява, като попълва заглавие, описание, цена, количество, координати и интервал за получаване.

3) Всяка обява може да бъде редактирана, деактивирана или изтрита, ако няма активни резервации към нея.

4) Магазинът следи входящите резервации в секцията „Резервации“.

5) При получаване на пакета въвежда кода, показан от клиента, и маркира поръчката като изпълнена.



5.3 Проведени тестове

№	Тестов сценарий	Очакван резултат	Статус
1	Регистрация и вход с валидни данни	Създава се акаунт и се осъществява вход	Успешен
2	Избор на роля след първи вход	Създава се профил в users	Успешен
3	Създаване на обява с валидни данни	Обявата се записва и се вижда в таблото	Успешен
4	Опит за създаване с невалиден интервал	Показва се съобщение за грешка	Успешен
5	Резервиране на активна обява	Намалява количеството и се създава поръчка	Успешен
6	Отказ на активна резервация	Статусът става cancelled и количеството се връща	Успешен
7	Потвърждение с правилен код	Поръчката става completed	Успешен
8	Търсене и филтриране на оферти	Списъкът се обновява според условията	Успешен

Тестовите са проведени ръчно чрез последователно изпълнение на основните сценарии. Проверени са също обработката на грешки при празни или невалидни полета, ограниченията за редакция при налични активни резервации и поведението при изтекли обяви.



6. Приложение

В настоящата глава са представени програмни фрагменти, които илюстрират основни аспекти от реализацията на проекта. Целта не е да бъде показан целия код, а да се демонстрират ключови елементи от логиката на приложението, свързани с обработката на оферти, резервации и управлението на данните. Подбраните примери отразяват най-съществените функционалности на разработената система и допълват описаните в предходните глави.

6.1 Модел на резервация

Следният фрагмент представя модела на резервация в системата. Чрез изброимия тип `OrderStatus` се дефинират възможните състояния на една поръчка, а структурата `Order` описва основните данни, които се съхраняват за всяка направена резервация. Този модел служи като основа за проследяване на поръчката - от нейното създаване до приключване или отказ.

```
enum OrderStatus: String, Codable, CaseIterable {  
  
    case reserved  
    case completed  
    case cancelled  
}  
  
struct Order: Codable, Identifiable {  
    let id: String  
    let listingId: String  
    let customerId: String  
    let storeId: String  
    let totalPrice: Double  
    let status: OrderStatus  
    let pickupCode: String?  
}
```




6.2 Транзакционна логика при резервиране

Една от най-важните части в реализацията на приложението е обработката на резервация чрез транзакция. Използването на транзакционен механизъм гарантира, че при създаване на поръчка и едновременно намаляване на наличното количество няма да възникнат несъответствия в данните. Това е особено важно при работа с няколко потребители едновременно, когато повече от един клиент може да се опита да резервира една и съща оферта.

```
let result = try await db.runTransaction { transaction, errorPointer -> Any?
in
  let snapshot = try transaction.getDocument(listingRef)
  let latestListing = try snapshot.data(as: Listing.self)
  guard latestListing.isAvailableForReservation else { return nil }
  let newQuantity = latestListing.quantity - 1
  transaction.setData(orderData, forDocument: orderRef)
  transaction.updateData([
    "quantity": newQuantity,
    "isActive": newQuantity > 0
  ], forDocument: listingRef)
  return order
}
```

6.3 Модел на оферта

Следващият фрагмент представя примерна структура на модел за оферта. Той съдържа основните променливи, необходими за публикуване, визуализиране и управление на предложенията в системата. Чрез този модел се реализира връзката между магазина, конкретната хранителна оферта, нейната цена, наличност и времеви интервал за получаване.

```
struct Listing: Codable, Identifiable {
  let id: String
  let storeId: String
  let title: String
  let details: String
  let price: Double
```



```
let quantity: Int  
let pickupStart: Date  
let pickupEnd: Date  
let isActive: Bool  
}
```

6.4 Логика за отказ на резервация

В системата е реализирана и възможност клиентът да откаже направена резервация, когато тя все още е в състояние „резервирана“. При тази операция се актуализира статусът на поръчката и при необходимост се възстановява наличното количество на съответната оферта. По този начин се гарантира коректно поддържане на наличностите и последователност на данните в базата.

```
transaction.updateData([  
  "status": OrderStatus.cancelled.rawValue  
], forDocument: orderRef)  
transaction.updateData([  
  "quantity": restoredQuantity,  
  "isActive": restoredIsActive  
], forDocument: listingRef)
```



7. Заключение

В резултат от настоящата курсова работа беше разработено работещо мобилно приложение, насочено към намаляване на хранителните отпадъци чрез публикуване и резервиране на продукти от търговски обекти. Основната идея на разработката е да се създаде удобна софтуерна среда, в която магазини и крайни потребители могат лесно да взаимодействат, като по този начин се подпомага по-ефективното използване на наличната храна и се намаляват загубите на продукти, които все още са годни за консумация.

Потребителският интерфейс е реализиран чрез SwiftUI, което позволява създаването на ясен и удобен за работа интерфейс. Архитектурата MVVM осигурява добро разделение между визуалната част, бизнес логиката и управлението на данните, което прави приложението по-лесно за поддръжка, разширяване и тестване. За автентикация и съхранение на данни са използвани Firebase Authentication и Cloud Firestore, които предоставят облачна инфраструктура за управление на потребители, обяви и резервации. Допълнително е използван Core Location, чрез който се реализира работа с географски координати и визуализация на близки обяви спрямо местоположението на потребителя.

Поставените в началото цели могат да се считат за изпълнени. Реализирани са основните функционалности на системата, включително регистрация и вход на потребители, разграничаване на роли „клиент“ и „магазин“, създаване, редактиране, активиране, деактивиране и изтриване на обяви, търсене и филтриране на налични предложения, резервиране и отказ на поръчки, както и потвърждаване на получаването чрез код. Системата поддържа и логика за скриване на невалидни или изтекли оферти, което допринася за по-реалистично поведение на приложението.



8. Използвана литература

- [1] SwiftUI - developer.apple.com/documentation/swiftui
- [2] Firebase Authentication - firebase.google.com/docs/auth/ios/start
- [3] Cloud Firestore - firebase.google.com/docs/firestore/quickstart и firebase.google.com/docs/firestore/manage-data/add-data
- [4] Cloud Firestore Security Rules - firebase.google.com/docs/firestore/security/get-started
- [5] Apple Developer Documentation Core Location - developer.apple.com/documentation/corelocation
- [6] Too Good To Go - toogoodtogo.com/en-gb/how-does-the-app-work
- [7] Olio - olioapp.com/en/
- [8] Karma - karma.life
- [9] Sommerville, I. Software Engineering. 10th ed. Pearson, 2015